Linguaggi di Programmazione

Fondamenti: cenni su computabilità

Roberto Gorrieri

Alma Mater Studiorum - Università di Bologna

Fondamenti: cenni su computabilità

Il problema della correttezza

- La **semantica statica** permette di fare controlli di correttezza sul testo del programma, senza mandarlo in esecuzione;
- la **semantica dinamica** permette di analizzare eventuali errori a runtime.

Domanda: possiamo rilevare tutti i possibili errori (statici o dinamici) mediante un analizzatore statico, cioè senza eseguire il programma?
Ovvero: può esistere un compilatore che, staticamente, rileva tutti i possibili errori, anche quelli che si verificheranno a runtime?
Ma anche, volendo essere più generosi: può esistere un programma che, anche eventualmente eseguendo parzialmente il programma in input, rileva in tempo finito tutti i possibili errori a runtime?

In altre parole

Esiste un programma che calcola *check*?

$$check(P) = \begin{cases} 1 & \text{se P è corretto} \\ 0 & \text{se P presenta errori} \end{cases}$$

Vediamo un caso specifico, in cui l'errore è la non-terminazione: se il programma è scritto in un linguaggio sequenziale, ci si aspetta che il suo calcolo termini sempre.

Dato un linguaggio di programmazione $\mathcal{L}$, proviamo a scrivere in $\mathcal{L}$ un programma H che calcola la seguente funzione:

$$H(P, x) = \begin{cases} 1 & \text{se } P(x) \downarrow (\text{"termina"}) \\ 0 & \text{se } P(x) \uparrow (\text{"diverge"}) \end{cases}$$

Per rispondere 0, il programma H deve riconoscere in tempo finito che P con input x non terminerà mai il calcolo. Può esistere un H siffatto?

Questo problema è chiamato Problema della fermata (Halting problem)

Halting problem

- Supponiamo per assurdo che H esista.
- Allora, usando H, possiamo realizzare l'applicazione

$$K(P) = \begin{cases} 1 & \text{se } P(P)\downarrow \\ 0 & \text{se } P(P)\uparrow \end{cases} = H(P,P)$$

Nota che con $H(P,P)$ affermiamo che P opera su una copia di sé stesso, usata come dato: situazione non inusuale (pensa ai compilatori che possono usare una copia di sé stessi come input).
Se H esiste, esiste anche K!

- Se esiste K, allora possiamo scrivere un programma G che prende in input un programma P e calcola

$$G(P) = \begin{cases} 1 & \text{se } K(P) = 0 \\ \uparrow & \text{se } K(P) = 1 \end{cases}$$

Se K esiste, allora anche G è facilmente programmabile.
Ma cosa succede se a G diamo in input G?

Halting problem (2)

Abbiamo due possibilità:

- $G(G) = 1$ sse $K(G) = 0$ sse $G(G) \uparrow$
- $G(G) = \uparrow$ sse $K(G) = 1$ sse $G(G) \downarrow$

Entrambe portano ad una contraddizione: Assurdo! Quindi, G non può esistere, e quindi K non può esistere, e quindi H non può esistere.

La funzione H è il primo esempio di funzione non calcolabile (o problema non risolvibile) - Alan Turing 1936.

Allora molte altre applicazioni non esistono!

$$Z(P) = \begin{cases} 1 & \text{se } P \text{ calcola la funzione costante 0: } \forall x\, P(x) = 0 \\ 0 & \text{altrimenti} \end{cases}$$

Z non è calcolabile perché, se esistesse tale programma, allora potrei costruire un programma per K! Come? Costruiamo un'applicazione

$$F(P)(x) = \begin{cases} 0 & \text{se } P(P)\downarrow \\ \uparrow & \text{se } P(P)\uparrow \end{cases}$$

Nota che $F(P)$ è calcolabile (o è la costante zero, oppure è la funzione ovunque divergente). Costruiamo $K(P) = Z(F(P))$. Infatti

$$Z(F(P)) = \begin{cases} 1 & \text{se } \forall x\, F(P)(x) = 0 \\ 0 & \text{altrimenti} \end{cases} = \begin{cases} 1 & \text{se } P(P)\downarrow \\ 0 & \text{se } P(P)\uparrow \end{cases} = K(P)$$

Dato che K non è calcolabile, neppure Z lo è!

Problema dell'equivalenza di programmi

$$Equiv(P, Q) = \begin{cases} 1 & \text{se } \forall x\, P(x) = Q(x) \\ 0 & \text{altrimenti} \end{cases}$$

Equiv non è calcolabile. Se lo fosse, potrei calcolare Z, cioè costruire un programma per Z, usando la funzione costante *Zero*:

$$Z(P) = Equiv(P, Zero) = \begin{cases} 1 & \text{se } \forall x\, P(x) = Zero(x) = 0 \\ 0 & \text{altrimenti} \end{cases}$$

Poiché già sappiamo che non può esistere un programma per Z, possiamo concludere che non può esistere un programma per *Equiv*!

(In)decidibilità

Possiamo risolvere questi problemi considerando un diverso linguaggio di programmazione?

Nota che nel ragionamento fatto, abbiamo solo assunto che il calcolo possa divergere (quindi che il linguaggio $\mathcal{L}$ abbia un costrutto di iterazione indeterminata o la ricorsione) e che un programma possa essere usato come dato per un altro programma.

Se il linguaggio è abbastanza ricco (cioè Turing-completo), allora NO!

Il problema della fermata (halting problem), per i normali linguaggi di programmazione, è **indecidibile**: non esiste alcuna procedura che lo risolva.

Procedura di decisione

Una procedura di decisione è una procedura che

- funziona per argomenti arbitrari,
- risponde "sì" o "no" in un tempo finito.

Esempio: $w \in L(G)$? È decidibile per, ad esempio, le grammatiche regolari.

Una problema è indecidibile se non esiste nessuna procedura di decisione che lo risolva. Ad esempio, il problema della fermata.

Procedura di semi-decisione

Una procedura di semi-decisione è una procedura che

- funziona per argomenti arbitrari.
- risponde "sì" in un tempo finito,
- non è in grado di rispondere "no" in tempo finito.

Esempio:

$$H'(P, x) = \begin{cases} 1 & \text{se } P(x) \downarrow \\ \uparrow & \text{se } P(x) \uparrow \end{cases}$$

La maggior parte dei problemi interessanti per i linguaggi di programmazione sono **indecidibili** (a volte semi-decidibili, come per il problema della fermata, ma poco utili in pratica).

Proprietà Indecidibili

- Terminazione

$$H(P,x) = \begin{cases} 1 \text{ se } P(x)\downarrow \\ 0 \text{ se } P(x)\uparrow \end{cases}$$

  è però semidecidibile!
- Divergenza

$$D(P,x) = \begin{cases} 1 \text{ se } P(x)\uparrow \\ 0 \text{ se } P(x)\downarrow \end{cases}$$

  non è nemmeno semidecidibile!
- Equivalenza di programmi
- Calcolo di una costante (cioè determinare se il programma calcola una funzione costante)
- Generazione di errori a runtime
- ...

Se consideriamo linguaggi a limitato potere espressivo, allora alcune di queste proprietà possono essere decise.

Esempio di linguaggio con limitato potere espressivo

Se un linguaggio di programmazione $\mathcal{L}$ non ha `while` (iterazione indeterminata), nè ricorsione, allora tutti i suoi programmi terminano:
$\Rightarrow$ il problema della fermata è decidibile, visto che la funzione $H(P, x) = 1$ per ogni P e per ogni x.

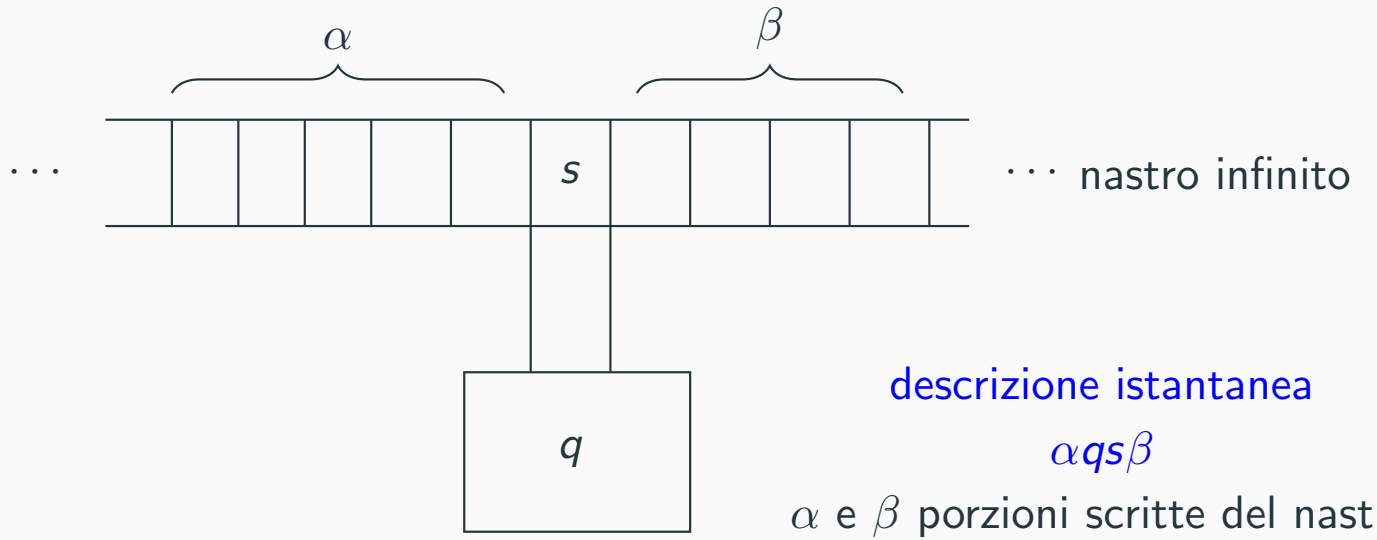
Ma se un linguaggio $\mathcal{L}$ è Turing-completo, allora le proprietà sopra sono tutte indecidibili.

Ma cosa vuol dire Turing-completo? Per rispondere, dobbiamo prima introdurre la Macchina di Turing (o Automa di Turing, a seconda dell'uso che se ne fa).

Macchina di Turing

Una Macchina di Turing (deterministica) è una sestupla $M = (Q, A, B, \delta, q_0, q_f)$ dove

- Q è un insieme finito di stati,
- A è l'alfabeto finito di input (tipicamente le cifre 0-9),
- B è l'alfabeto finito del nastro ($A \subset B, \square \in B$),
- q_0 è lo stato iniziale,
- q_f è lo stato finale,
- $\delta : Q \times B \multimap Q \times B \times \{sx, dx\}$ tale che $\delta(q_f, b)$ è indefinita $\forall b \in B$.



descrizione istantanea

$\alpha qs\beta$

α e β porzioni scritte del nastro

Mosse di una Macchina di Turing

Mosse di una Macchina di Turing:

- $\alpha q s \beta \vdash \alpha s' q' \beta$ se $\delta(q, s) = (q', s', dx)$
- $\alpha \underline{s} q s \beta \vdash \alpha q' \underline{s} s' \beta$ se $\delta(q, s) = (q', s', sx)$

Il linguaggio riconosciuto dalla Macchina/Automa di Turing M è:

$L[M] = \{w \in A^* | q_0 w \vdash^* \alpha q_f \beta\}$

ma la M.d.T., partendo da q_0 per in input w, può non raggiungere mai nè q_f, nè uno stato di blocco "erroneo", cioè può divergere.

Funzione calcolata da una Macchina di Turing

Una Macchina di Turing deterministica M su alfabeto $A = [0-9]$ può essere vista come una macchina che calcola funzioni parziali binarie: $f_M : \mathbb{N} \multimap \{0,1\}$

$$f_M(w) = \begin{cases} 1 & \text{se } q_0 w \vdash^* \alpha q_f \beta \\ 0 & \text{se } q_0 w \vdash^* \alpha q' \beta \nvdash \text{ e } q' \neq q_f \\ \uparrow & \text{altrimenti} \end{cases}$$

dove $w \in [0-9]^+$, ovvero $w \in \mathbb{N}$. Se consideriamo l'insieme di tutte le funzioni parziali a valori binari

$$\mathbb{F} = \{f | f : \mathbb{N} \multimap \{0,1\}\}$$

possiamo concludere che $f \in \mathbb{F}$ è Turing-calcolabile se esiste una M.d.T. M tale che $f_M = f$.

Funzioni calcolate dalle Macchine di Turing

Una Macchina di Turing deterministica M calcola la funzione f_M. Allora come sarà l'insieme di tutte le funzioni Turing-calcolabili?

$$\mathbb{F}_{MdT} = \{f \in \mathbb{F} \mid \exists \text{ MdT } M.\ f = f_M\}$$

Domanda: $\mathbb{F}_{MdT}$ coincide con $\mathbb{F}$? O esistono funzioni in $\mathbb{F}$ che non sono Turing-calcolabili?

Risposta: La maggior parte delle funzioni in $\mathbb{F}$ non sono Turing-calcolabili!!! Cioè $\mathbb{F}_{MdT} \subsetneq \mathbb{F}$.

Argomento di Cantor

Supponiamo che le funzioni binarie (consideriamo solo le totali, per semplicità) siano enumerabili. Dimostreremo che è impossibile.

	0	1	2	3	4	$\cdots$
f_1	0	0	1	1	1	$\cdots$
f_2	0	1	0	1	0	$\cdots$
f_3	1	0	0	0	0	$\cdots$
f_4	0	1	1	1	0	$\cdots$
f_5	1	1	1	0	0	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Definiamo la funzione $\overline{f}_d(i) = \overline{f_i(i)}$ cioè se $f_i(i) = 1$ allora $\overline{f}_d(i) = 0$ e viceversa.

Per ogni $i \in \mathbb{N}$, $\overline{f}_d(i) \neq f_i(i)$, cioè $\overline{f}_d$ non compare nell'elenco!

In realtà, si può dimostrare che l'insieme $\mathbb{F}$ ha la cardinalità dell'insieme dei numeri reali $\mathbb{R}$, quindi più grande della cardinalità di $\mathbb{N}$.

Quindi...

D'altro lato, le M.d.T. sono enumerabili: $M_0, M_1, M_2, ...$, quindi di M.d.T. ce ne sono tante quante $\mathbb{N}$.

Di conseguenza, la cardinalità di $\mathbb{F}_{MdT}$ è pari a quella di $\mathbb{N}$. Perciò, la stragrande maggioranza delle funzioni in $\mathbb{F}$ non è calcolabile!

Osservazione: analogia con l'insieme di tutti i linguaggi formali e l'insieme di tutti i linguaggi rappresentabili da grammatiche: $\mathcal{P}(A^*) \cong \mathbb{R}$ mentre l'insieme dei linguaggi rappresentabili con le grammatiche $\cong \mathbb{N}$, già visto ad inizio corso.

Turing-completezza

Formalismo Turing-completo: formalismo che ha la stessa potenza espressiva delle M.d.T., cioè calcola le stesse funzioni calcolabili con M.d.T.

Negli anni '30, '40, '50, '60 del secolo scorso sonostati proposti numerosi formalismi:

- M.d.T.,
- λ-calcolo,
- funzioni generali ricorsive,
- Random Access Machines (RAM),
- Counter Machines
- ...

Tutti dimostrati equivalenti tra loro (cioè tutti Turing-completi) attraverso compilazione di un formalismo in un altro.

E i linguaggi di programmazione? Tutti Turing-completi, purché sia possibile usare tutta la memoria di cui necessitano (un computer ha solo memoria finita, per cui, a rigore, non può essere equivalente ad una M.d.T.)

Teorema di Böhm-Jacopini (1966)

Un linguaggio di programmazione imperativo che contenga

- `if-then-else` (condizionale)
- `while` (iterazione indeterminata)
- `;` (composizione sequenziale)

(+ l'operazione base di assegnamento) è Turing-completo.

N.B.: Il linguaggio usato per spiegare la semantica dinamica con tecnica SOS in Lez05 è Turing-completo.

Osservazione: Questo teorema ha avuto un benefico effetto nel promuovere la "programmazione strutturata", cioè il principio secondo il quale un linguaggio dovrebbe contenere solo operatori ben strutturati, la cui semantica sia definibile "localmente", solo guardando i propri argomenti (non è così, ad esempio, per l'istruzione `goto`).

Tesi di Church-Turing (1937)

Tesi di Church-Turing
Se una funzione matematica può essere calcolata algoritmicamente in qualche formalismo, allora è calcolabile con M.d.T.

È una **tesi** (e non un teorema) perché non può essere dimostrata. Infatti,

- Non c'è una definizione formale di cosa è "algoritmicamente" calcolabile; inoltre
- È implicita una quantificazione universale su tutti i possibili formalismi! Infinite prove? E se domani arriva un nuovo formalismo?

Considerata vera perché in 90 anni nessuno è riuscito a confutarla!

Criterio di equivalenza tra linguaggi sequenziali: se $\mathcal{L}_1$ e $\mathcal{L}_2$ sono entrambi Turing-completi, allora sono equamente espressivi per la tesi di Church-Turing.

N.B.: Nel caso dei linguaggi concorrenti, la questione è più delicata, perché i programmi concorrenti non calcolano solo funzioni, ma risolvono problemi, offrono servizi, ecc...

Gerarchia di Macchine: Espressività vs Analizzabilità

Espressività vs Analizzabilità

Nel caso più generale:

M.d.T $\leftrightarrow$ Grammatiche generali$\leftrightarrow$ $w \in L(M)/L(G)$? semidecidibile

Ma abbiamo visto formalismi più deboli, comunque significativi:

PDA $\leftrightarrow$ grammatiche libere $\leftrightarrow$ $w \in L[N](L(G)$ problema decidibile

Anche se il problema dell'equivalenza "$L[N_1] = L[N_2]$?" è indecidibile (ma è decidibile per DPDA)

DFA $\leftrightarrow$ grammatiche regolari $\leftrightarrow$ $L[N_1] = L[N_2]$? decidibile

Osservazione: circuiti logici, vending machines, funzioni find/replace nei word processor, algoritmi di verifica di correttezza delle password,... sono DFA! Quindi esistono molte "funzioni" interessanti che si possono calcolare in formalismi **non** Turing-completi! E su questi si possono decidere molte proprietà interessanti!